Fake News Detection: Training with Tinker

Fine-tunes a supported LLM via Tinker LoRA on the LIAR2 dataset.

Run order: 01 → 02 → this notebook

1. Setup

```
import sys, os, json
sys.path.insert(0, '../src')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import accuracy_score, f1_score,
confusion_matrix, classification_report
from sklearn.utils import resample
from collections import defaultdict
from dotenv import load_dotenv
load_dotenv('../.env')

from config import TRUTHFULNESS_LABELS, RANDOM_SEED, TINKER_CONFIG,
PROMPT_TEMPLATE
from models import TinkerClassifier, ClassBalancer
from data_processor import DataProcessor
from training_utils import TinkerTrainer

np.random.seed(RANDOM_SEED)
print('Imports OK')
print(f'Base model : {TINKER_CONFIG["base_model"]}')
print(f'LoRA rank  : {TINKER_CONFIG["lora_rank"]}')
print(f'Epochs     : {TINKER_CONFIG["epochs"]}')
print(f'Batch size : {TINKER_CONFIG["batch_size"]}')
```

```
Imports OK
Base model : nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B-BF16
LoRA rank  : 16
Epochs     : 3
Batch size : 8
```

```
# Connect to Tinker and create LoRA training client.
# Reads TINKER_API_KEY from .env automatically.
# Allocates a GPU on Tinker infrastructure — takes a few seconds.
classifier = TinkerClassifier(
    base_model=TINKER_CONFIG['base_model'],
    prompt_template=PROMPT_TEMPLATE,
)
```

```
classifier.connect()
classifier.create_training_client(lora_rank=TINKER_CONFIG['lora_rank']
)

tokenizer = classifier.get_tokenizer()
print('Tinker training client ready')
print(classifier.get_model_info())
```

```
INFO:models:TinkerClassifier initialized with
base_model=nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B-BF16
INFO:models:Connecting to Tinker...
INFO:tinker.lib.public_interfaces.service_client:ServiceClient
initialized for session 5794487f-aa33-5c03-8e09-f30e7d43d60a
INFO:models:Connected to Tinker service
INFO:models:Creating LoRA training client (rank=16,
model=nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B-BF16)...
INFO:tinker.lib.public_interfaces.service_client:TrainingClient
initialized for model 5794487f-aa33-5c03-8e09-f30e7d43d60a:train:0
INFO:models:Training client ready
Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending
unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
enable higher rate limits and faster downloads.

Tinker training client ready
{'base_model': 'nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B-BF16', 'type':
'TinkerClassifier (LoRA)', 'labels': ['barely-true', 'false', 'half-
true', 'mostly-true', 'pants-fire', 'true'], 'training_client':
'active', 'sampling_client': 'none'}
```

2. Load & Prepare Data

```
# Tinker needs raw natural-language text, not word-index sequences.
# Load the cleaned dataset from Phase 1.
artifacts_dir = '../artifacts'
df = pd.read_csv(f'{artifacts_dir}/processed_data.csv')
print(f'Loaded {len(df)} rows')

# Use original statement text so the LLM sees natural language.
texts  = df['statement'].astype(str).tolist()
labels = df['label'].astype(str).str.lower().str.strip().tolist()

unknown = set(labels) - set(TinkerClassifier.LABELS)
if unknown:
    print(f'WARNING: unknown labels: {unknown}')
```

```
else:
    print(f'All labels valid: {sorted(set(labels))}')
```

```
Loaded 10240 rows
All labels valid: ['barely-true', 'false', 'half-true', 'mostly-true',
'pants-fire', 'true']
```

```
# Oversample minority classes, then do a 70/15/15 split.
label_indices = defaultdict(list)
for i, lbl in enumerate(labels):
    label_indices[lbl].append(i)

max_count = max(len(v) for v in label_indices.values())
balanced_indices = []
for lbl, idxs in label_indices.items():
    resampled = resample(idxs, n_samples=max_count, replace=True,
                         random_state=RANDOM_SEED)
    balanced_indices.extend(resampled)

np.random.shuffle(balanced_indices)
texts_bal  = [texts[i]  for i in balanced_indices]
labels_bal = [labels[i] for i in balanced_indices]
print(f'Balanced: {len(texts_bal)} samples ({max_count} per class)')

n       = len(texts_bal)
n_train = int(n * 0.70)
n_val   = int(n * 0.15)

texts_train  = texts_bal[:n_train]
labels_train = labels_bal[:n_train]
texts_val    = texts_bal[n_train : n_train + n_val]
labels_val   = labels_bal[n_train : n_train + n_val]
texts_test   = texts_bal[n_train + n_val:]
labels_test  = labels_bal[n_train + n_val:]
print(f'Split  -> Train:{len(texts_train)}  Val:{len(texts_val)}
Test:{len(texts_test)}')
```

```
Balanced: 12684 samples (2114 per class)
Split  -> Train:8878  Val:1902  Test:1904
```

```
# Tokenise each example into a Tinker Datum.
# Prompt tokens get weight 0 (not trained), completion tokens get
weight 1.
print('Building training datums...', flush=True)
train_datums = DataProcessor.prepare_tinker_dataset(
    texts_train, labels_train, tokenizer, PROMPT_TEMPLATE)

print('Building validation datums...', flush=True)
val_datums = DataProcessor.prepare_tinker_dataset(
    texts_val, labels_val, tokenizer, PROMPT_TEMPLATE)
```

```
print(f'Done. Train={len(train_datums)}  Val={len(val_datums)}',
flush=True)

# Quick sanity check
d = train_datums[0]
print(f'Example final weights : {list(d.loss_fn_inputs["weights"])[-
6:]}')
```

```
Building training datums...

INFO:data_processor:Prepared 8878 Tinker datums

Building validation datums...

INFO:data_processor:Prepared 1902 Tinker datums

Done. Train=8878  Val=1902
Example final weights : [('data', [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1]), ('dtype', 'float32'),
('shape', None)]
```

3. Tinker LoRA Training

```
trainer = TinkerTrainer(
    classifier=classifier,
    learning_rate=TINKER_CONFIG['learning_rate'],
    batch_size=TINKER_CONFIG['batch_size'],
)

history = trainer.train(
    train_datums=train_datums,
    val_datums=val_datums,
    epochs=TINKER_CONFIG['epochs'],
)

print('Training history:')
for ep, (tl, vl) in enumerate(
        zip(history['train_loss'], history['val_loss']), 1):
    print(f'  Epoch {ep}: train={tl:.4f}  val={vl:.4f}')
```

```
INFO:training_utils:Starting Tinker training: 3 epochs, 8878 train /
1902 val datums, batch_size=8, lr=0.0001
INFO:training_utils:
Epoch 1/3
INFO:training_utils:  Batch 10/1110  loss=0.9294
INFO:training_utils:  Batch 20/1110  loss=0.8377
INFO:training_utils:  Batch 30/1110  loss=0.9340
INFO:training_utils:  Batch 40/1110  loss=0.7804
INFO:training_utils:  Batch 50/1110  loss=0.7403
```

```
INFO:training_utils:  Batch 60/1110  loss=0.6174
INFO:training_utils:  Batch 70/1110  loss=0.9202
INFO:training_utils:  Batch 80/1110  loss=0.7641
INFO:training_utils:  Batch 90/1110  loss=0.8120
INFO:training_utils:  Batch 100/1110  loss=0.7244
INFO:training_utils:  Batch 110/1110  loss=0.9194
INFO:training_utils:  Batch 120/1110  loss=0.8135
INFO:training_utils:  Batch 130/1110  loss=0.5269
INFO:training_utils:  Batch 140/1110  loss=0.6598
INFO:training_utils:  Batch 150/1110  loss=0.9751
INFO:training_utils:  Batch 160/1110  loss=0.6658
INFO:training_utils:  Batch 170/1110  loss=0.9264
INFO:training_utils:  Batch 180/1110  loss=0.7326
INFO:training_utils:  Batch 190/1110  loss=0.7015
INFO:training_utils:  Batch 200/1110  loss=0.5739
INFO:training_utils:  Batch 210/1110  loss=0.9836
INFO:training_utils:  Batch 220/1110  loss=0.6399
INFO:training_utils:  Batch 230/1110  loss=0.8098
INFO:training_utils:  Batch 240/1110  loss=0.5628
INFO:training_utils:  Batch 250/1110  loss=0.6872
INFO:training_utils:  Batch 260/1110  loss=1.0110
INFO:training_utils:  Batch 270/1110  loss=0.7765
INFO:training_utils:  Batch 280/1110  loss=0.5182
INFO:training_utils:  Batch 290/1110  loss=0.4299
INFO:training_utils:  Batch 300/1110  loss=0.7910
INFO:training_utils:  Batch 310/1110  loss=0.6539
INFO:training_utils:  Batch 320/1110  loss=0.6789
INFO:training_utils:  Batch 330/1110  loss=0.4436
INFO:training_utils:  Batch 340/1110  loss=0.8191
INFO:training_utils:  Batch 350/1110  loss=0.6958
INFO:training_utils:  Batch 360/1110  loss=0.6699
INFO:training_utils:  Batch 370/1110  loss=0.6904
INFO:training_utils:  Batch 380/1110  loss=0.6400
INFO:training_utils:  Batch 390/1110  loss=0.6501
INFO:training_utils:  Batch 400/1110  loss=0.5967
INFO:training_utils:  Batch 410/1110  loss=0.5729
INFO:training_utils:  Batch 420/1110  loss=0.6469
INFO:training_utils:  Batch 430/1110  loss=0.7133
INFO:training_utils:  Batch 440/1110  loss=0.6210
INFO:training_utils:  Batch 450/1110  loss=0.5272
INFO:training_utils:  Batch 460/1110  loss=0.6402
INFO:training_utils:  Batch 470/1110  loss=0.7748
INFO:training_utils:  Batch 480/1110  loss=0.5273
INFO:training_utils:  Batch 490/1110  loss=0.7885
INFO:training_utils:  Batch 500/1110  loss=0.5302
INFO:training_utils:  Batch 510/1110  loss=0.5267
INFO:training_utils:  Batch 520/1110  loss=0.6128
INFO:training_utils:  Batch 530/1110  loss=0.6684
INFO:training_utils:  Batch 540/1110  loss=0.6135
```

```
INFO:training_utils:  Batch 550/1110  loss=0.4022
INFO:training_utils:  Batch 560/1110  loss=0.5431
INFO:training_utils:  Batch 570/1110  loss=0.5557
INFO:training_utils:  Batch 580/1110  loss=0.5841
INFO:training_utils:  Batch 590/1110  loss=0.6068
INFO:training_utils:  Batch 600/1110  loss=0.7054
INFO:training_utils:  Batch 610/1110  loss=0.5729
INFO:training_utils:  Batch 620/1110  loss=0.6240
INFO:training_utils:  Batch 630/1110  loss=1.0604
INFO:training_utils:  Batch 640/1110  loss=0.5222
INFO:training_utils:  Batch 650/1110  loss=0.6444
INFO:training_utils:  Batch 660/1110  loss=0.9642
INFO:training_utils:  Batch 670/1110  loss=0.5920
INFO:training_utils:  Batch 680/1110  loss=0.5956
INFO:training_utils:  Batch 690/1110  loss=0.4241
INFO:training_utils:  Batch 700/1110  loss=0.5433
INFO:training_utils:  Batch 710/1110  loss=0.3024
INFO:training_utils:  Batch 720/1110  loss=0.6094
INFO:training_utils:  Batch 730/1110  loss=0.5918
INFO:training_utils:  Batch 740/1110  loss=0.9609
INFO:training_utils:  Batch 750/1110  loss=0.5097
INFO:training_utils:  Batch 760/1110  loss=0.4104
INFO:training_utils:  Batch 770/1110  loss=0.5008
INFO:training_utils:  Batch 780/1110  loss=0.2363
INFO:training_utils:  Batch 790/1110  loss=0.8350
INFO:training_utils:  Batch 800/1110  loss=0.5338
INFO:training_utils:  Batch 810/1110  loss=0.5119
INFO:training_utils:  Batch 820/1110  loss=0.4483
INFO:training_utils:  Batch 830/1110  loss=0.4686
INFO:training_utils:  Batch 840/1110  loss=0.6853
INFO:training_utils:  Batch 850/1110  loss=0.8163
INFO:training_utils:  Batch 860/1110  loss=0.5386
INFO:training_utils:  Batch 870/1110  loss=0.8714
INFO:training_utils:  Batch 880/1110  loss=0.5662
INFO:training_utils:  Batch 890/1110  loss=0.6281
INFO:training_utils:  Batch 900/1110  loss=0.2818
INFO:training_utils:  Batch 910/1110  loss=0.5225
INFO:training_utils:  Batch 920/1110  loss=0.4522
INFO:training_utils:  Batch 930/1110  loss=0.1848
INFO:training_utils:  Batch 940/1110  loss=0.3328
INFO:training_utils:  Batch 950/1110  loss=0.2401
INFO:training_utils:  Batch 960/1110  loss=0.6520
INFO:training_utils:  Batch 970/1110  loss=0.7097
INFO:training_utils:  Batch 980/1110  loss=0.5760
INFO:training_utils:  Batch 990/1110  loss=0.5553
INFO:training_utils:  Batch 1000/1110  loss=0.5168
INFO:training_utils:  Batch 1010/1110  loss=0.4442
INFO:training_utils:  Batch 1020/1110  loss=0.5242
INFO:training_utils:  Batch 1030/1110  loss=0.3918
```

```
INFO:training_utils:  Batch 1040/1110  loss=0.5063
INFO:training_utils:  Batch 1050/1110  loss=0.7049
INFO:training_utils:  Batch 1060/1110  loss=0.4207
INFO:training_utils:  Batch 1070/1110  loss=0.4405
INFO:training_utils:  Batch 1080/1110  loss=0.5221
INFO:training_utils:  Batch 1090/1110  loss=0.5796
INFO:training_utils:  Batch 1100/1110  loss=0.6421
INFO:training_utils:  Batch 1110/1110  loss=0.3304
INFO:training_utils:  Train loss: 0.6426  |  Val loss: 0.4395
INFO:training_utils:
Epoch 2/3
INFO:training_utils:  Batch 10/1110  loss=0.1012
INFO:training_utils:  Batch 20/1110  loss=0.0542
INFO:training_utils:  Batch 30/1110  loss=0.1549
INFO:training_utils:  Batch 40/1110  loss=0.0676
INFO:training_utils:  Batch 50/1110  loss=0.4915
INFO:training_utils:  Batch 60/1110  loss=0.0720
INFO:training_utils:  Batch 70/1110  loss=0.2159
INFO:training_utils:  Batch 80/1110  loss=0.1605
INFO:training_utils:  Batch 90/1110  loss=0.2894
INFO:training_utils:  Batch 100/1110  loss=0.2261
INFO:training_utils:  Batch 110/1110  loss=0.2816
INFO:training_utils:  Batch 120/1110  loss=0.1786
INFO:training_utils:  Batch 130/1110  loss=0.3134
INFO:training_utils:  Batch 140/1110  loss=0.2989
INFO:training_utils:  Batch 150/1110  loss=0.0347
INFO:training_utils:  Batch 160/1110  loss=0.1913
INFO:training_utils:  Batch 170/1110  loss=0.1905
INFO:training_utils:  Batch 180/1110  loss=0.2849
INFO:training_utils:  Batch 190/1110  loss=0.1629
INFO:training_utils:  Batch 200/1110  loss=0.2319
INFO:training_utils:  Batch 210/1110  loss=0.4099
INFO:training_utils:  Batch 220/1110  loss=0.1943
INFO:training_utils:  Batch 230/1110  loss=0.0944
INFO:training_utils:  Batch 240/1110  loss=0.2465
INFO:training_utils:  Batch 250/1110  loss=0.3012
INFO:training_utils:  Batch 260/1110  loss=0.2994
INFO:training_utils:  Batch 270/1110  loss=0.1869
INFO:training_utils:  Batch 280/1110  loss=0.0325
INFO:training_utils:  Batch 290/1110  loss=0.5558
INFO:training_utils:  Batch 300/1110  loss=0.0924
INFO:training_utils:  Batch 310/1110  loss=0.1595
INFO:training_utils:  Batch 320/1110  loss=0.1165
INFO:training_utils:  Batch 330/1110  loss=0.3114
INFO:training_utils:  Batch 340/1110  loss=0.1359
INFO:training_utils:  Batch 350/1110  loss=0.5289
INFO:training_utils:  Batch 360/1110  loss=0.1330
INFO:training_utils:  Batch 370/1110  loss=0.3492
INFO:training_utils:  Batch 380/1110  loss=0.1083
```

```
INFO:training_utils:  Batch 390/1110  loss=0.0335
INFO:training_utils:  Batch 400/1110  loss=0.1497
INFO:training_utils:  Batch 410/1110  loss=0.0671
INFO:training_utils:  Batch 420/1110  loss=0.2465
INFO:training_utils:  Batch 430/1110  loss=0.1334
INFO:training_utils:  Batch 440/1110  loss=0.1842
INFO:training_utils:  Batch 450/1110  loss=0.1318
INFO:training_utils:  Batch 460/1110  loss=0.0825
INFO:training_utils:  Batch 470/1110  loss=0.0999
INFO:training_utils:  Batch 480/1110  loss=0.0196
INFO:training_utils:  Batch 490/1110  loss=0.0662
INFO:training_utils:  Batch 500/1110  loss=0.1063
INFO:training_utils:  Batch 510/1110  loss=0.0455
INFO:training_utils:  Batch 520/1110  loss=0.1450
INFO:training_utils:  Batch 530/1110  loss=0.1260
INFO:training_utils:  Batch 540/1110  loss=0.1426
INFO:training_utils:  Batch 550/1110  loss=0.2165
INFO:training_utils:  Batch 560/1110  loss=0.0885
INFO:training_utils:  Batch 570/1110  loss=0.1068
INFO:training_utils:  Batch 580/1110  loss=0.1746
INFO:training_utils:  Batch 590/1110  loss=0.3137
INFO:training_utils:  Batch 600/1110  loss=0.3672
INFO:training_utils:  Batch 610/1110  loss=0.0977
INFO:training_utils:  Batch 620/1110  loss=0.1391
INFO:training_utils:  Batch 630/1110  loss=0.0647
INFO:training_utils:  Batch 640/1110  loss=0.2173
INFO:training_utils:  Batch 650/1110  loss=0.1133
INFO:training_utils:  Batch 660/1110  loss=0.1167
INFO:training_utils:  Batch 670/1110  loss=0.1662
INFO:training_utils:  Batch 680/1110  loss=0.1535
INFO:training_utils:  Batch 690/1110  loss=0.0327
INFO:training_utils:  Batch 700/1110  loss=0.3214
INFO:training_utils:  Batch 710/1110  loss=0.1277
INFO:training_utils:  Batch 720/1110  loss=0.0077
INFO:training_utils:  Batch 730/1110  loss=0.4171
INFO:training_utils:  Batch 740/1110  loss=0.1701
INFO:training_utils:  Batch 750/1110  loss=0.0592
INFO:training_utils:  Batch 760/1110  loss=0.3493
INFO:training_utils:  Batch 770/1110  loss=0.1449
INFO:training_utils:  Batch 780/1110  loss=0.1983
INFO:training_utils:  Batch 790/1110  loss=0.1369
INFO:training_utils:  Batch 800/1110  loss=0.3540
INFO:training_utils:  Batch 810/1110  loss=0.0429
INFO:training_utils:  Batch 820/1110  loss=0.1391
INFO:training_utils:  Batch 830/1110  loss=0.1951
INFO:training_utils:  Batch 840/1110  loss=0.1228
INFO:training_utils:  Batch 850/1110  loss=0.0679
INFO:training_utils:  Batch 860/1110  loss=0.1192
INFO:training_utils:  Batch 870/1110  loss=0.3593
```

```
INFO:training_utils:  Batch 880/1110  loss=0.0828
INFO:training_utils:  Batch 890/1110  loss=0.3556
INFO:training_utils:  Batch 900/1110  loss=0.1541
INFO:training_utils:  Batch 910/1110  loss=0.0191
INFO:training_utils:  Batch 920/1110  loss=0.2176
INFO:training_utils:  Batch 930/1110  loss=0.0384
INFO:training_utils:  Batch 940/1110  loss=0.3203
INFO:training_utils:  Batch 950/1110  loss=0.1471
INFO:training_utils:  Batch 960/1110  loss=0.3889
INFO:training_utils:  Batch 970/1110  loss=0.0357
INFO:training_utils:  Batch 980/1110  loss=0.1061
INFO:training_utils:  Batch 990/1110  loss=0.1294
INFO:training_utils:  Batch 1000/1110  loss=0.2021
INFO:training_utils:  Batch 1010/1110  loss=0.1254
INFO:training_utils:  Batch 1020/1110  loss=0.1955
INFO:training_utils:  Batch 1030/1110  loss=0.0584
INFO:training_utils:  Batch 1040/1110  loss=0.1743
INFO:training_utils:  Batch 1050/1110  loss=0.3638
INFO:training_utils:  Batch 1060/1110  loss=0.1403
INFO:training_utils:  Batch 1070/1110  loss=0.1113
INFO:training_utils:  Batch 1080/1110  loss=0.1654
INFO:training_utils:  Batch 1090/1110  loss=0.0200
INFO:training_utils:  Batch 1100/1110  loss=0.0299
INFO:training_utils:  Batch 1110/1110  loss=0.0468
INFO:training_utils:  Train loss: 0.1854  |  Val loss: 0.4084
INFO:training_utils:
Epoch 3/3
INFO:training_utils:  Batch 10/1110  loss=0.0057
INFO:training_utils:  Batch 20/1110  loss=0.0043
INFO:training_utils:  Batch 30/1110  loss=0.0044
INFO:training_utils:  Batch 40/1110  loss=0.0033
INFO:training_utils:  Batch 50/1110  loss=0.0011
INFO:training_utils:  Batch 60/1110  loss=0.0067
INFO:training_utils:  Batch 70/1110  loss=0.0209
INFO:training_utils:  Batch 80/1110  loss=0.0185
INFO:training_utils:  Batch 90/1110  loss=0.0002
INFO:training_utils:  Batch 100/1110  loss=0.0299
INFO:training_utils:  Batch 110/1110  loss=0.0049
INFO:training_utils:  Batch 120/1110  loss=0.0025
INFO:training_utils:  Batch 130/1110  loss=0.0008
INFO:training_utils:  Batch 140/1110  loss=0.0050
INFO:training_utils:  Batch 150/1110  loss=0.0023
INFO:training_utils:  Batch 160/1110  loss=0.0011
INFO:training_utils:  Batch 170/1110  loss=0.0103
INFO:training_utils:  Batch 180/1110  loss=0.0273
INFO:training_utils:  Batch 190/1110  loss=0.0008
INFO:training_utils:  Batch 200/1110  loss=0.0002
INFO:training_utils:  Batch 210/1110  loss=0.0013
INFO:training_utils:  Batch 220/1110  loss=0.0091
```

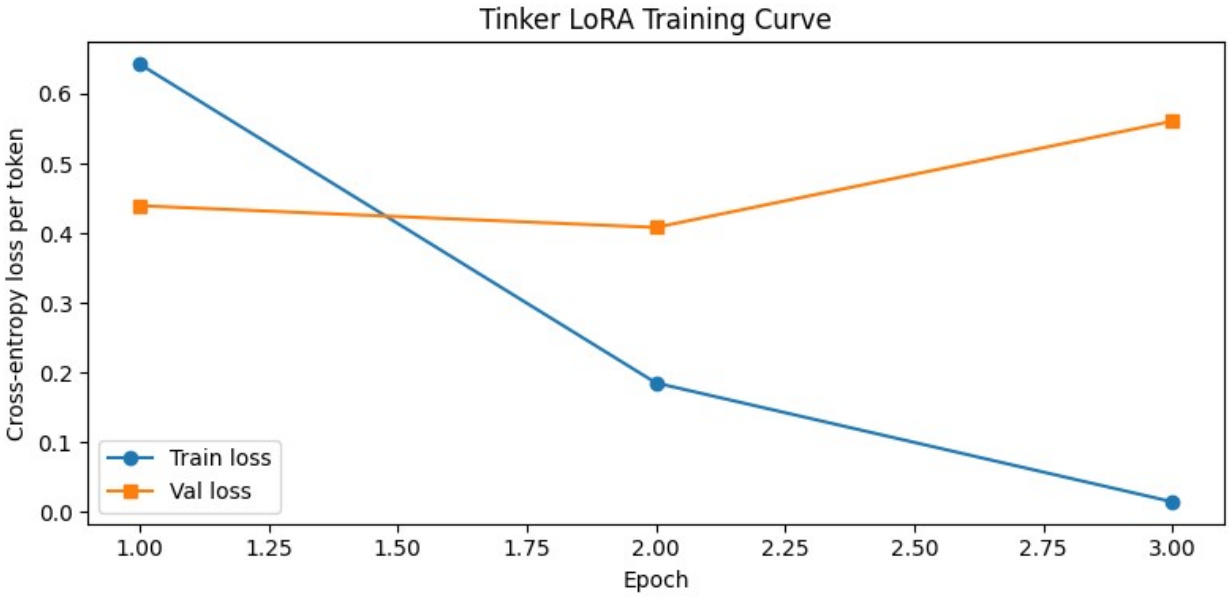
```
INFO:training_utils:  Batch 230/1110  loss=0.0003
INFO:training_utils:  Batch 240/1110  loss=0.0113
INFO:training_utils:  Batch 250/1110  loss=0.0025
INFO:training_utils:  Batch 260/1110  loss=0.0012
INFO:training_utils:  Batch 270/1110  loss=0.0021
INFO:training_utils:  Batch 280/1110  loss=0.0044
INFO:training_utils:  Batch 290/1110  loss=0.0013
INFO:training_utils:  Batch 300/1110  loss=0.0039
INFO:training_utils:  Batch 310/1110  loss=0.0002
INFO:training_utils:  Batch 320/1110  loss=0.0011
INFO:training_utils:  Batch 330/1110  loss=0.0112
INFO:training_utils:  Batch 340/1110  loss=0.0015
INFO:training_utils:  Batch 350/1110  loss=0.0011
INFO:training_utils:  Batch 360/1110  loss=0.0225
INFO:training_utils:  Batch 370/1110  loss=0.0032
INFO:training_utils:  Batch 380/1110  loss=0.0036
INFO:training_utils:  Batch 390/1110  loss=0.0812
INFO:training_utils:  Batch 400/1110  loss=0.0024
INFO:training_utils:  Batch 410/1110  loss=0.0001
INFO:training_utils:  Batch 420/1110  loss=0.0009
INFO:training_utils:  Batch 430/1110  loss=0.0010
INFO:training_utils:  Batch 440/1110  loss=0.0139
INFO:training_utils:  Batch 450/1110  loss=0.0042
INFO:training_utils:  Batch 460/1110  loss=0.0131
INFO:training_utils:  Batch 470/1110  loss=0.0305
INFO:training_utils:  Batch 480/1110  loss=0.0011
INFO:training_utils:  Batch 490/1110  loss=0.0003
INFO:training_utils:  Batch 500/1110  loss=0.0043
INFO:training_utils:  Batch 510/1110  loss=0.0010
INFO:training_utils:  Batch 520/1110  loss=0.0018
INFO:training_utils:  Batch 530/1110  loss=0.0007
INFO:training_utils:  Batch 540/1110  loss=0.0008
INFO:training_utils:  Batch 550/1110  loss=0.0003
INFO:training_utils:  Batch 560/1110  loss=0.0009
INFO:training_utils:  Batch 570/1110  loss=0.0005
INFO:training_utils:  Batch 580/1110  loss=0.0468
INFO:training_utils:  Batch 590/1110  loss=0.0002
INFO:training_utils:  Batch 600/1110  loss=0.2264
INFO:training_utils:  Batch 610/1110  loss=0.0008
INFO:training_utils:  Batch 620/1110  loss=0.0282
INFO:training_utils:  Batch 630/1110  loss=0.0004
INFO:training_utils:  Batch 640/1110  loss=0.0008
INFO:training_utils:  Batch 650/1110  loss=0.0029
INFO:training_utils:  Batch 660/1110  loss=0.0048
INFO:training_utils:  Batch 670/1110  loss=0.0009
INFO:training_utils:  Batch 680/1110  loss=0.0052
INFO:training_utils:  Batch 690/1110  loss=0.0069
INFO:training_utils:  Batch 700/1110  loss=0.0035
INFO:training_utils:  Batch 710/1110  loss=0.0036
```

```
INFO:training_utils:  Batch 720/1110  loss=0.0012
INFO:training_utils:  Batch 730/1110  loss=0.0002
INFO:training_utils:  Batch 740/1110  loss=0.0007
INFO:training_utils:  Batch 750/1110  loss=0.0005
INFO:training_utils:  Batch 760/1110  loss=0.0016
INFO:training_utils:  Batch 770/1110  loss=0.0002
INFO:training_utils:  Batch 780/1110  loss=0.0012
INFO:training_utils:  Batch 790/1110  loss=0.0219
INFO:training_utils:  Batch 800/1110  loss=0.0012
INFO:training_utils:  Batch 810/1110  loss=0.0003
INFO:training_utils:  Batch 820/1110  loss=0.0380
INFO:training_utils:  Batch 830/1110  loss=0.0002
INFO:training_utils:  Batch 840/1110  loss=0.1330
INFO:training_utils:  Batch 850/1110  loss=0.0002
INFO:training_utils:  Batch 860/1110  loss=0.0003
INFO:training_utils:  Batch 870/1110  loss=0.0049
INFO:training_utils:  Batch 880/1110  loss=0.0005
INFO:training_utils:  Batch 890/1110  loss=0.0122
INFO:training_utils:  Batch 900/1110  loss=0.0001
INFO:training_utils:  Batch 910/1110  loss=0.0007
INFO:training_utils:  Batch 920/1110  loss=0.0010
INFO:training_utils:  Batch 930/1110  loss=0.0036
INFO:training_utils:  Batch 940/1110  loss=0.0023
INFO:training_utils:  Batch 950/1110  loss=0.0001
INFO:training_utils:  Batch 960/1110  loss=0.0002
INFO:training_utils:  Batch 970/1110  loss=0.0001
INFO:training_utils:  Batch 980/1110  loss=0.0114
INFO:training_utils:  Batch 990/1110  loss=0.0012
INFO:training_utils:  Batch 1000/1110  loss=0.0004
INFO:training_utils:  Batch 1010/1110  loss=0.0055
INFO:training_utils:  Batch 1020/1110  loss=0.0007
INFO:training_utils:  Batch 1030/1110  loss=0.0084
INFO:training_utils:  Batch 1040/1110  loss=0.0049
INFO:training_utils:  Batch 1050/1110  loss=0.0046
INFO:training_utils:  Batch 1060/1110  loss=0.0002
INFO:training_utils:  Batch 1070/1110  loss=0.2074
INFO:training_utils:  Batch 1080/1110  loss=0.0007
INFO:training_utils:  Batch 1090/1110  loss=0.0018
INFO:training_utils:  Batch 1100/1110  loss=0.0015
INFO:training_utils:  Batch 1110/1110  loss=0.0307
INFO:training_utils:  Train loss: 0.0150  |  Val loss: 0.5607

Training history:
  Epoch 1: train=0.6426  val=0.4395
  Epoch 2: train=0.1854  val=0.4084
  Epoch 3: train=0.0150  val=0.5607
```

```
plt.figure(figsize=(8, 4))
ep = range(1, len(history['train_loss']) + 1)
plt.plot(ep, history['train_loss'], marker='o', label='Train loss')
```

```
plt.plot(ep, history['val_loss'],   marker='s', label='Val loss')
plt.xlabel('Epoch')
plt.ylabel('Cross-entropy loss per token')
plt.title('Tinker LoRA Training Curve')
plt.legend()
plt.tight_layout()
plt.show()
```



4. Save Weights

```
# Save LoRA weights to Tinker and get a sampling client.
# The URI is written to disk so predict.py can reload the model later.
tinker_uri = classifier.save_for_inference(name='fake-news-liar2')
print(f'Tinker URI: {tinker_uri}')

uri_path = f'{artifacts_dir}/tinker_weights_uri.txt'
with open(uri_path, 'w') as f:
    f.write(tinker_uri)
print(f'URI saved to {uri_path}')
```

```
INFO:models:Saving weights as 'fake-news-liar2'...
INFO:models:Weights saved. URI: nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B-
BF16

Tinker URI: nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B-BF16
URI saved to ../artifacts/tinker_weights_uri.txt
```

5. Test Set Evaluation

```
print(f'Predicting {len(texts_test)} test examples...')
test_preds = []
for i, text in enumerate(texts_test):
    pred = classifier.predict(text)
    test_preds.append(pred)
    if (i + 1) % 100 == 0:
        print(f'  {i + 1}/{len(texts_test)}')

label_order = TinkerClassifier.LABELS  # alphabetical
label2idx   = {l: i for i, l in enumerate(label_order)}
y_true = [label2idx.get(l, -1) for l in labels_test]
y_pred = [label2idx.get(l, -1) for l in test_preds]

test_acc = accuracy_score(y_true, y_pred)
test_f1  = f1_score(y_true, y_pred, average='macro', zero_division=0)
print(f'Test Accuracy : {test_acc:.4f}')
print(f'Test Macro F1 : {test_f1:.4f}')
```

```
Predicting 1904 test examples...
  100/1904
  200/1904
  300/1904
  400/1904
  500/1904
  600/1904
  700/1904
  800/1904
  900/1904
  1000/1904
  1100/1904
  1200/1904
  1300/1904
  1400/1904
  1500/1904
  1600/1904
  1700/1904
  1800/1904
  1900/1904
Test Accuracy : 0.7190
Test Macro F1 : 0.7203
```

6. Detailed Evaluation

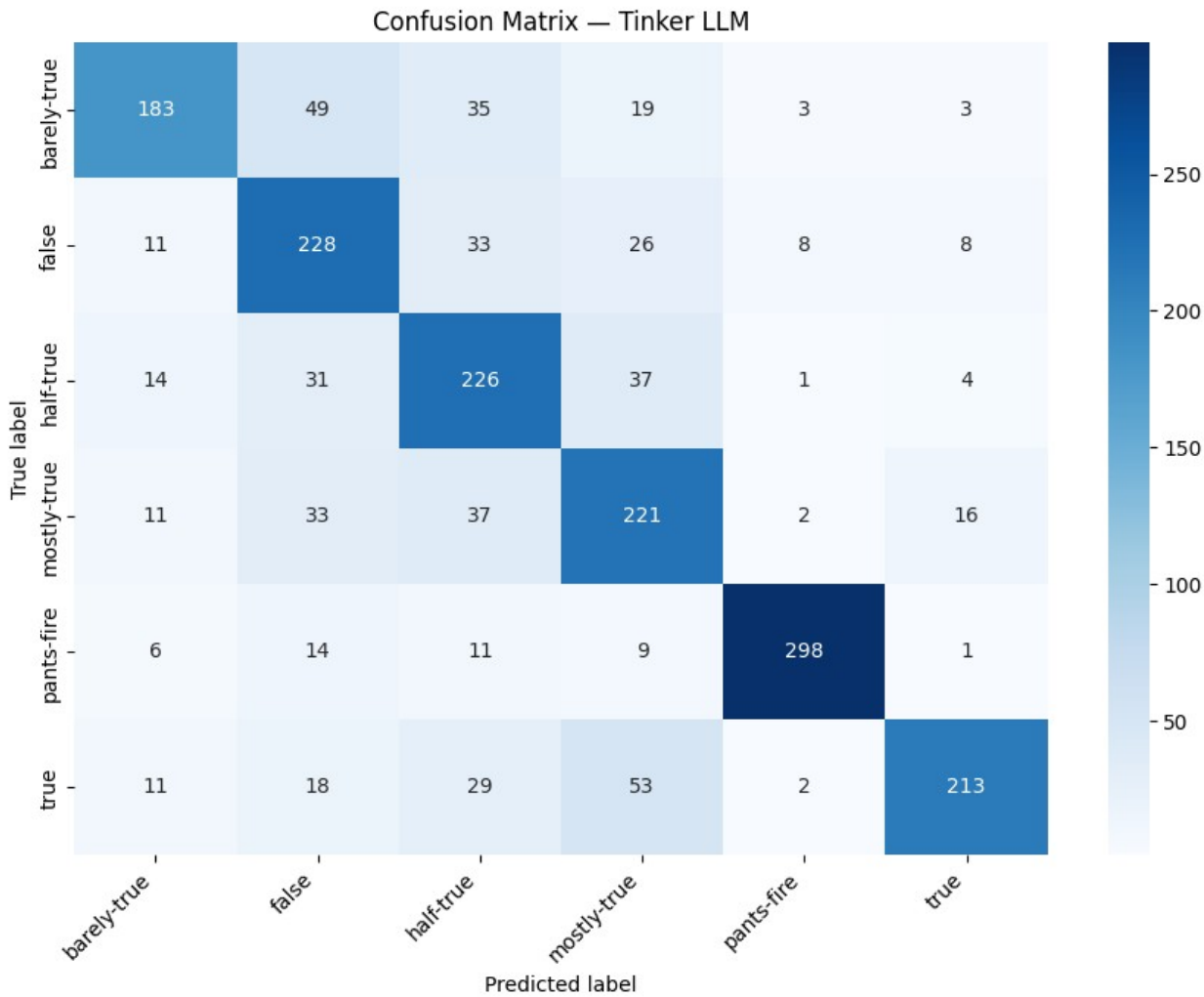
```
print(classification_report(y_true, y_pred,
                            target_names=label_order,
zero_division=0))
```

```
              precision    recall  f1-score   support

 barely-true       0.78      0.63      0.69       292
       false       0.61      0.73      0.66       314
   half-true       0.61      0.72      0.66       313
 mostly-true       0.61      0.69      0.65       320
  pants-fire       0.95      0.88      0.91       339
        true       0.87      0.65      0.75       326

    accuracy                           0.72      1904
   macro avg       0.74      0.72      0.72      1904
weighted avg       0.74      0.72      0.72      1904
```

```
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(9, 7))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=label_order, yticklabels=label_order)
plt.title('Confusion Matrix — Tinker LLM')
plt.ylabel('True label')
plt.xlabel('Predicted label')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



7. Save Results

```
results = {
    'tinker_llm': {
        'base_model':        TINKER_CONFIG['base_model'],
        'lora_rank':         TINKER_CONFIG['lora_rank'],
        'tinker_uri':        tinker_uri,
        'test_accuracy':     float(test_acc),
        'test_macro_f1':     float(test_f1),
        'train_loss_history': history['train_loss'],
        'val_loss_history':   history['val_loss'],
    }
}
with open(f'{artifacts_dir}/training_results.json', 'w') as f:
    json.dump(results, f, indent=2)
print('Saved training_results.json')
```

```
Saved training_results.json
```

8. Final Summary

```
print('=' * 60)
print('TRAINING COMPLETE')
print('=' * 60)
print(f'  Model      : {TINKER_CONFIG["base_model"]}')
print(f'  LoRA rank  : {TINKER_CONFIG["lora_rank"]}')
print(f'  Test acc   : {test_acc:.4f}')
print(f'  Test F1    : {test_f1:.4f}')
print(f'  URI        : {tinker_uri}')
print('=' * 60)
print()
print('To run inference:')
print('  python src/predict.py --text "Your statement here" --proba')
```

```
============================================================
TRAINING COMPLETE
============================================================
  Model      : nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B-BF16
  LoRA rank  : 16
  Test acc   : 0.7190
  Test F1    : 0.7203
  URI        : nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B-BF16
============================================================

To run inference:
  python src/predict.py --text "Your statement here" --proba
```